

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf\_ae3ec722-12a\agent-a8210328e56b4c4fd.jsonl`
- SHA-256: `34eaf8c06588a27b3fbac2b8c8652b8acd4ffe8d971256c6903b2bfd2c40b739`
- Source modified: `2026-07-06T07:54:21+00:00`
- Imported at: `2026-07-08T16:00:08+00:00`
- project: `wf\_ae3ec722-12a`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T07:52:24.500Z)

Reviewing GitHub PR #37 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-c-delivery-routing -> main, one commit 100abb2).
It implements ADR 0011 Track C (docs/adr/0011-provider-neutral-domain-schema.md): populates & uses the destinations / routing_rules / delivery_attempts tables that Track B (#35, merged) created empty.
Changed files ONLY: src/tg_video_filter/db.py, src/tg_video_filter/delivery.py, src/tg_video_filter/models.py, tests/test_db.py, tests/test_delivery.py. Diff +675/-9.

Key new code:

- db.py: `_migration_004_destinations_and_routing` (backfills destinations from `filter_config_deprecated`, `routing_rules` per source, `delivery_attempts` from `videos_deprecated.forwarded=1`); `repos` `upsert_destination`, `get_destination`, `upsert_delivery_attempt`, `record_delivery` (UPSERT with `attempt_count` increment), `get_delivery_attempt`.
- delivery.py: `forward_to_target/send_link_message/deliver` gain an optional `destination_id` kwarg; when set, `record_delivery` writes a `DeliveryAttempt`; when None (all current live callers) falls back to the `mark_forwarded` no-op.
- models.py: `Destination`, `RoutingRule`, `DeliveryStatus`, `DeliveryAttempt`.

Note: routing is NOT yet wired into the live path (`telethon_client/manager` still call `deliver()` with `destination_id=None`); `destination_id` is currently exercised only by tests. So `record_delivery` issues are LATENT until Track D, but ship now.

Ground rules:

- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run a repro: `C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>`. Build schema via `db._apply_schema(conn)` on `aiosqlite ":memory:"`; or `db.init_db(path)` on a temp file seeded with a legacy v1 schema (`users/channels/videos/filter_config`) to exercise the migration ladder (001->004). NOTE: a legacy 'videos' table MUST include columns `duration_s,width,height,size_bytes,mime_type` or migration 002's column-copy fails (that's a test-harness requirement, not a bug).
- Authoritative local gate ALREADY RUN (don't re-run full suite): `ruff check clean`; `pytest 451` passed @ 95.75% (floor 80%); `ruff format --check flags commands.py` ONLY because this branch predates the merged #36 format fix (`commands.py` is NOT in this PR) ? do NOT report that as a #37 finding.

- Report ONLY defects in THIS PR's diff. Distinguish LIVE-NOW (migration runs on upgrade) from LATENT (delivery recording, unused until Track D). No praise. Concrete failure scenario per finding. Rank severe first.
- Severity: blocker (data loss/corruption/wrong core behaviour on upgrade), high (real bug, narrow trigger), medium (latent-but-real logic bug), low (hygiene/edge). Be honest; downgrade unrealistic triggers.

DIMENSION: delivery.py + record_delivery correctness (delivery.py:127-235; db.py record_delivery ~1310).

Investigate:

- SEED to verify/refute (CONFIRMED by reviewer repro): BOTH mode calls forward_to_target then send_link_message with the SAME destination_id. If forward succeeds and link fails, the shared delivery_attempts row ends status=FAILED with delivered_at set (the failed link masks the successful forward); attempt_count also over-counts (2 for one logical delivery). A future retry/idempotency consumer reading status=failed would re-deliver an already-forwarded video or misreport. Reproduce and assess.
- record_delivery UPSERT: attempt_count=delivery_attempts.attempt_count+1 on conflict, delivered_at kept on failure ? verify increment & delivered_at semantics are correct.
- The failure-recording branch (elif destination_id is not None: record_delivery(success=False)) ? delivery.py coverage shows lines 145,152,181,224 UNCOVERED. Is the failure path correct AND untested?
- destination_id threading through deliver() for LINK/BOTH/FORWARD; backwards-compat when destination_id=None.

Return findings per schema.

2. assistant (2026-07-06T07:52:27.652Z)

I'll review PR #37's delivery.py and record_delivery correctness. Let me start by reading the key files.

3. user (2026-07-06T07:52:28.632Z)

```
1      """Delivery: send matched videos to the user's Saved Messages.
2
3      Three delivery modes (per-user, configured via FilterConfig.delivery_mode):
4
5      - ``FORWARD``: forward the video itself (re-uploads to Saved Messages).
6      - ``LINK``: send a text message with a t.me deep link + metadata summary.
7      - ``BOTH``: forward the video AND send the link message.
8
9      All modes wrap their Telethon calls with ``FloodWaitError`` handling (sleep
10     ``seconds + jitter``, retry once) and mark the DB row ``forwarded=1`` on success.
11     """
12
13     from __future__ import annotations
14
15     import asyncio
16     import logging
17     import random
18     from collections.abc import Awaitable, Callable
19
20     import aiosqlite
21     from telethon import TelegramClient
22     from telethon.errors import FloodWaitError
23     from telethon.tl.types import Message
24
25     from tg_video_filter import db
26     from tg_video_filter.models import DeliveryMode
27
```

```

28     log = logging.getLogger(__name__)
29
30     # Extra jitter (seconds) added on top of the requested flood wait so we don't
31     # retry at the exact boundary and risk re-triggering the limit.
32     _FLOOD_JITTER_RANGE = (1.0, 5.0)
33
34
35     # --- pure link-message formatting -----
36
37
38     def _format_duration(seconds: int | None) -> str:
39         """Human-readable duration for the link summary."""
40         if seconds is None:
41             return "?"
42         if seconds < 60:
43             return f"{seconds}s"
44         if seconds < 3600:
45             return f"{seconds // 60}min"
46         h = seconds // 3600
47         m = (seconds % 3600) // 60
48         return f"{h}h{m:02d}min" if m else f"{h}h"
49
50
51     def _format_size(size_bytes: int | None) -> str:
52         """Human-readable file size for the link summary."""
53         if size_bytes is None:
54             return "?"
55         mib = size_bytes / (1024 * 1024)
56         if mib >= 1024:
57             return f"{mib / 1024:.1f}GB"
58         return f"{mib:.0f}MB"
59
60
61     def _format_height(height: int | None) -> str:
62         """Resolution label: 1080 -> 1080p, None -> ?."""
63         if height is None:
64             return "?"
65         return f"{height}p"
66
67
68     def build_link_text(
69         *,
70         message_link: str | None,
71         duration_s: int | None,
72         height: int | None,
73         size_bytes: int | None,
74         channel_title: str | None,
75     ) -> str:
76         """Build the text message sent in LINK/BOTH mode.
77
78         Format::
79
80             ? 1080p · 5min · 480MB
81             #ChannelName
82             https://t.me/c/1234567890/42
83
84         ``message_link`` may be None if the link can't be constructed (e.g. the
85         channel is inaccessible); in that case the link line is omitted.
86         """
87         lines = [
88             f"? {_format_height(height)} · {_format_duration(duration_s)} · "
89             f"{_format_size(size_bytes)}"
90         ]
91         if channel_title:
92             # Sanitize: replace spaces in the title for a hashtag-like display.

```

```

92         lines.append(f"#{channel_title.replace(' ', '_')}")
93     if message_link:
94         lines.append(message_link)
95     return "\n".join(lines)
96
97
98     # --- Telethon interaction (retry/flood logic) -----
99
100
101     async def _send_with_retry(
102         client: TelegramClient,
103         send_callable: Callable[[], Awaitable[None]],
104         video_id: int,
105     ) -> bool:
106         """Execute a send/forward callable with FloodWait handling. Returns success."""
107         try:
108             await send_callable()
109         except FloodWaitError as e:
110             wait = e.seconds + random.uniform(*_FLOOD_JITTER_RANGE)
111             log.warning("flood_wait video=%s seconds=%s retrying_after=%.1f", video_id,
e.seconds, wait)
112             await asyncio.sleep(wait)
113             try:
114                 await send_callable()
115             except Exception:
116                 log.exception("delivery_retry_failed video=%s", video_id)
117                 return False
118         except Exception:
119             log.exception("delivery_failed video=%s", video_id)
120             return False
121         return True
122
123
124     async def forward_to_target(
125         client: TelegramClient,
126         message: Message,
127         conn: aiosqlite.Connection,
128         video_id: int,
129         target: str = "me",
130         *,
131         destination_id: int | None = None,
132     ) -> bool:
133         """Forward ``message`` to ``target``, recording delivery state.
134
135         ``target`` is a Telethon entity: ``"me"`` (Saved Messages), ``"@username"`` ,
136         or a numeric chat id string. See ADR 0005.
137
138         If ``destination_id`` is provided, a per-destination ``DeliveryAttempt``
139         is written (ADR 0011 Track C). Otherwise falls back to the legacy no-op
140         ``mark_forwarded``.
141         """
142         ok = await _send_with_retry(client, lambda: client.forward_messages(target,
message), video_id)
143         if ok:
144             if destination_id is not None:
145                 await db.record_delivery(
146                     conn, content_item_id=video_id, destination_id=destination_id,
success=True
147                 )
148             else:
149                 await db.mark_forwarded(conn, video_id)
150                 log.info("video_forwarded video=%s target=%s", video_id, target)
151         elif destination_id is not None:
152             await db.record_delivery(
153                 conn, content_item_id=video_id, destination_id=destination_id, success=False

```

```

154         )
155     return ok
156
157
158     async def send_link_message(
159         client: TelegramClient,
160         text: str,
161         conn: aiomysqlite.Connection,
162         video_id: int,
163         target: str = "me",
164         *,
165         destination_id: int | None = None,
166     ) -> bool:
167         """Send a link/summary text message to ``target``, recording delivery state.
168
169         ``target`` is a Telethon entity; see :func:`forward_to_target`.
170         """
171         ok = await _send_with_retry(client, lambda: client.send_message(target, text),
video_id)
172         if ok:
173             if destination_id is not None:
174                 await db.record_delivery(
175                     conn, content_item_id=video_id, destination_id=destination_id,
success=True
176                 )
177             else:
178                 await db.mark_forwarded(conn, video_id)
179                 log.info("link_sent video=%s target=%s", video_id, target)
180         elif destination_id is not None:
181             await db.record_delivery(
182                 conn, content_item_id=video_id, destination_id=destination_id, success=False
183             )
184         return ok
185
186
187     async def deliver(
188         client: TelegramClient,
189         message: Message,
190         conn: aiomysqlite.Connection,
191         video_id: int,
192         mode: DeliveryMode,
193         *,
194         link_text: str | None = None,
195         target: str = "me",
196         destination_id: int | None = None,
197     ) -> bool:
198         """Dispatch delivery based on ``mode`` to ``target``.
199
200         ``link_text`` is required for LINK/BOTH modes (built by the caller via
201         :func:`build_link_text`). If it's None in LINK/BOTH mode, falls back to
202         FORWARD so we never silently drop a matched video.
203
204         ``target`` is a Telethon entity (``"me"`` , ``"@username"`` , chat id).
205         See ADR 0005.
206
207         ``destination_id`` is the FK to ``destinations.id`` (ADR 0011 Track C).
208         When provided, a per-destination ``DeliveryAttempt`` is written.
209         """
210         if mode == DeliveryMode.LINK and link_text is not None:
211             return await send_link_message(
212                 client, link_text, conn, video_id, target, destination_id=destination_id
213             )
214         if mode == DeliveryMode.BOTH and link_text is not None:
215             fwd_ok = await forward_to_target(
216                 client, message, conn, video_id, target, destination_id=destination_id

```

```

217         )
218         link_ok = await send_link_message(
219             client, link_text, conn, video_id, target, destination_id=destination_id
220         )
221         if not link_ok:
222             log.warning("link_delivery_failed_in_both_mode video=%s", video_id)
223         if not fwd_ok:
224             log.warning("forward_delivery_failed_in_both_mode video=%s", video_id)
225         return fwd_ok or link_ok
226     # FORWARD, or fallback when link_text is missing.
227     if mode != DeliveryMode.FORWARD:
228         log.warning(
229             "link_mode_fallback_to_forward video=%s mode=%s link_text_missing=%s",
230             video_id,
231             mode.value,
232             link_text is None,
233         )
234     return await forward_to_target(
235         client, message, conn, video_id, target, destination_id=destination_id
236     )
237
238
239     __all__ = [
240         "build_link_text",
241         "deliver",
242         "forward_to_target",
243         "send_link_message",
244     ]
245

```

4. user (2026-07-06T07:52:28.692Z)

```

1     """Pydantic data models for users, channels, videos, and filter config.
2
3     These models are the in-memory representation used by the repository layer
4     (`db.py`) and the filter engine (`filters/`). They are deliberately
5     decoupled from Telethon's own types so the filter engine stays pure and
6     unit-testable without a live Telegram connection.
7     """
8
9     from __future__ import annotations
10
11     from datetime import UTC, datetime
12     from enum import Enum
13
14     from pydantic import BaseModel, ConfigDict, Field, field_validator, model_validator
15
16
17     def _utcnow() -> datetime:
18         """Timezone-aware UTC 'now', used as a default factory."""
19         return datetime.now(UTC)
20
21
22     class DeliveryMode(str, Enum):
23         """How a matched video is delivered to the user.
24
25         - ``FORWARD``: forward the video itself to Saved Messages (re-uploads).
26         - ``LINK``: send a text message with a t.me link + metadata summary.
27         - ``BOTH``: forward the video AND send the link message.
28         """
29
30         FORWARD = "forward"
31         LINK = "link"
32         BOTH = "both"
33

```

```

34
35 class Account(BaseModel):
36     """A product identity ? the owner of config, channels, routes, and content.
37
38     Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39     is the bridge used by the Bot API front-door (``from.id``) and by the
40     legacy userbot path; it is nullable so future providers that don't have a
41     Telegram id (e.g. WhatsApp) can still own an ``Account``.
42     """
43
44     model_config = ConfigDict(frozen=True)
45
46     id: int = Field(description="Stable surrogate PK; the owner key for all downstream
tables")
47     telegram_id: int | None = Field(
48         default=None,
49         description="Telegram user id (bot from.id OR userbot account id). UNIQUE when
set.",
50     )
51     provider: str = Field(default="telegram", description="Provider namespace; future:
'whatsapp'")
52     display_name: str | None = None
53     created_at: datetime = Field(default_factory=_utcnow)
54
55
56 class TelegramSession(BaseModel):
57     """An optional MTPROTO (Telethon) session linked to an :class:`Account`.
58
59     A bot-only user has an ``Account`` row but no ``TelegramSession`` row;
60     that is what makes them structurally invisible to ``manager.run``
61     (ADR 0010). ``session_path`` is data-driven so the file-location
62     convention is explicit rather than derived from the PK at runtime.
63     """
64
65     model_config = ConfigDict(frozen=True)
66
67     account_id: int = Field(description="FK ? accounts.id; PK (one session per account
in v1)")
68     phone: str | None = Field(default=None, description="E.164 phone; used only for
bootstrap")
69     session_path: str = Field(description="File-backed Telethon session path
(user_<id>.session)")
70     enabled: bool = Field(
71         default=True,
72         description="If False, the manager skips this session (not the account)",
73     )
74     created_at: datetime = Field(default_factory=_utcnow)
75     last_login_at: datetime | None = None
76
77
78 class User(BaseModel):
79     """Deprecated: legacy v1 model conflating identity + MTPROTO session.
80
81     Retained as a backwards-compatible view over the new ``accounts`` +
82     ``telegram_sessions`` tables (ADR 0010). New code should use
83     :class:`Account` + :class:`TelegramSession` directly. The ``db`` layer
84     maps ``User`` ? (``Account``, ``TelegramSession``) so existing callers
85     and tests keep working without a flag day.
86     """
87
88     model_config = ConfigDict(frozen=True)
89
90     id: int = Field(description="Telegram user id (== accounts.telegram_id for legacy
rows)")
91     phone: str | None = Field(default=None, description="E.164 phone; used only for

```

```

bootstrap")
92     enabled: bool = Field(default=True, description="If False, the manager skips this
session")
93     created_at: datetime = Field(default_factory=_utcnow)
94
95
96     class Channel(BaseModel):
97         """A channel/group a user is a member of. Populated lazily as messages arrive.
98
99         Deprecated: a provider-neutral view over :class:`Source` (ADR 0011). New
100         code should use :class:`Source` directly; the ``db`` layer maps
101         ``Channel`` ? ``Source`` so existing callers and tests keep working.
102         """
103
104         model_config = ConfigDict(frozen=True)
105
106         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
107         user_id: int
108         channel_id: int = Field(description="Telegram channel/chat id")
109         title: str | None = None
110         seen_first_at: datetime = Field(default_factory=_utcnow)
111
112
113     # --- provider-neutral domain models (ADR 0011, Track B) -----
114     #
115     # These generalize the legacy v1 ``Channel`` / ``VideoRecord`` / ``FilterConfig``
116     # shapes so the schema no longer leaks Telegram-specific ids into every future
117     # provider's code path. ``provider`` columns + provider-prefixed ``dedup_key``
118     # make cross-provider collisions structurally impossible. See ADR 0011 for the
119     # full rationale and the two decisions (Option 1A: tiered ``dedup_key``;
120     # Option 2A: ``policies`` generalizes ``filter_config``).
121
122
123     class Source(BaseModel):
124         """A configured place we ingest content from (ADR 0011).
125
126         Generalizes the legacy :class:`Channel` (1:1 for Telegram today; future
127         providers add their own source kinds). ``provider_source_id`` is the
128         provider-native id (Telegram ``channel_id``, future WhatsApp group id, ?).
129         """
130
131         model_config = ConfigDict(frozen=True)
132
133         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
134         account_id: int = Field(description="FK ? accounts.id (owner of the source)")
135         provider: str = Field(default="telegram", description="Provider namespace")
136         provider_source_id: str = Field(
137             description="Provider-native source id (Telegram channel_id, ?)"
138         )
139         title: str | None = None
140         kind: str = Field(default="channel", description="'channel' | 'group' | 'feed'")
141         watched: bool = Field(
142             default=False,
143             description="ADR 0009 selective scanning: ingest from this source iff True",
144         )
145         seen_first_at: datetime = Field(default_factory=_utcnow)
146         last_seen_at: datetime | None = None
147
148
149     class ContentItem(BaseModel):
150         """A normalized content record discovered from any :class:`Source` (ADR 0011).
151
152         Generalizes the legacy :class:`VideoRecord`. Phase 1 stores only videos
153         (``content_type='video'``); ``text`` / ``link_url`` are landed now but stay
154         NULL until later tracks populate them (avoids a future ``ALTER TABLE``).

```



```

155     ``dedup_key`` is the provider-neutral dedup key (Option 1A: tiered lookup
156     wrapping ADR 0006, computed by :func:`tg_video_filter.db.compute_dedup_key`).
157     """
158
159     model_config = ConfigDict(frozen=True)
160
161     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
162     account_id: int = Field(description="FK ? accounts.id")
163     source_id: int = Field(description="FK ? sources.id")
164     provider: str = Field(default="telegram", description="Provider namespace")
165     provider_message_id: str = Field(
166         description="Provider-native message id (Telegram message_id, ?)"
167     )
168     content_type: str = Field(default="video", description="'video' | 'link' |
'article'")
169     text: str | None = Field(default=None, description="Caption / post text (nullable in
Phase 1)")
170     link_url: str | None = Field(default=None, description="For link/article posts
(Phase 1: NULL)")
171     media_ref: str | None = Field(default=None, description="Provider media reference")
172     document_id: str | None = Field(
173         default=None,
174         description="Telegram document.id (kept for TG-specific Tier-1 lookups)",
175     )
176     duration_s: int | None = None
177     width: int | None = None
178     height: int | None = None
179     size_bytes: int | None = None
180     mime_type: str | None = None
181     dedup_key: str = Field(description="Provider-neutral dedup key (Option 1A); unique
per account")
182     matched: bool = False
183     seen_at: datetime = Field(default_factory=_utcnow)
184
185
186     class Policy(BaseModel):
187         """A named, reusable filtering policy owned by an account (ADR 0011, Option 2A).
188
189         Generalizes the legacy per-account ``filter_config`` row so one account can
190         own multiple policies (enables ADR 0009's per-route filter overrides) and so
191         future enrichment/security stages grow into the same ``config_json`` column.
192         Exactly one policy per account carries ``is_default=True``; the legacy
193         ``load_filter_config`` / ``save_filter_config`` shims read/write that row.
194         """
195
196         model_config = ConfigDict(frozen=True)
197
198         id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
199         account_id: int = Field(description="FK ? accounts.id")
200         name: str = Field(description="Policy name; 'default' for the account default
policy")
201         config_json: str = Field(
202             description="FilterConfig JSON blob (Phase-1 shape); grows in Phase 2 (Track F)"
203         )
204         is_default: bool = Field(
205             default=False, description="Exactly one per account (UNIQUE partial index)"
206         )
207         created_at: datetime = Field(default_factory=_utcnow)
208         updated_at: datetime = Field(default_factory=_utcnow)
209
210
211     # --- Track C: destinations, routing, delivery state (ADR 0011) -----
212     #
213     # These tables were created empty by Track B (_SCHEMA); Track C populates
214     # destinatinons (backfilled from filter_config.delivery_target), writes

```

```

215 # delivery_attempts (replacing the legacy videos.forwarded boolean), and
216 # enforces the source/destination invariant (a channel is either source or
217 # destination, never both).
218
219
220 class Destination(BaseModel):
221     """A delivery target ? where matched content is sent (ADR 0011, Track C).
222
223     Generalizes the legacy ``FilterConfig.delivery_target`` field so one account
224     can have multiple named output channels. Backfilled from the existing
225     per-account ``delivery_target``; the default destination is the target used
226     when no explicit routing rule matches.
227     """
228
229     model_config = ConfigDict(frozen=True)
230
231     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
232     account_id: int = Field(description="FK ? accounts.id")
233     provider: str = Field(default="telegram", description="Provider namespace")
234     target: str = Field(description="Telethon entity: 'me', '@username', or chat id")
235     kind: str = Field(default="saved", description="'saved' | 'channel' | 'chat'")
236     title: str | None = None
237     is_output: bool = Field(
238         default=False,
239         description="ADR 0009: distinguishes output channels from ingest sources",
240     )
241     created_at: datetime = Field(default_factory=_utcnow)
242
243
244 class RoutingRule(BaseModel):
245     """A rule that maps a source through a policy to a destination (ADR 0009).
246
247     Canonical name: ``routing_rules`` (locked in umbrella issue #25). When
248     ``policy_id`` is NULL, the account's default policy is used. When multiple
249     rules match, the highest ``priority`` wins.
250     """
251
252     model_config = ConfigDict(frozen=True)
253
254     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
255     account_id: int = Field(description="FK ? accounts.id")
256     source_id: int = Field(description="FK ? sources.id")
257     policy_id: int | None = Field(
258         default=None,
259         description="FK ? policies.id; NULL means use account default policy",
260     )
261     destination_id: int = Field(description="FK ? destinations.id")
262     enabled: bool = Field(default=True)
263     priority: int = Field(default=0, description="Higher wins when multiple rules
match")
264     created_at: datetime = Field(default_factory=_utcnow)
265
266
267 class DeliveryStatus(str, Enum):
268     """Per-destination delivery state, replacing the legacy binary ``forwarded``."""
269
270     PENDING = "pending"
271     DELIVERED = "delivered"
272     FAILED = "failed"
273     RETRYING = "retrying"
274
275
276 class DeliveryAttempt(BaseModel):
277     """Per-destination delivery record ? idempotent via UNIQUE index.
278

```

```

279     Replaces the legacy ``videos.forwarded`` boolean (ADR 0011). One row per
280     ``(content_item_id, destination_id)`` pair. On retry, ``attempt_count`` is
281     incremented and ``status``/``last_error`` are updated.
282     """
283
284     model_config = ConfigDict(frozen=True)
285
286     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
287     content_item_id: int = Field(description="FK ? content_items.id")
288     destination_id: int = Field(description="FK ? destinations.id")
289     route_id: int | None = Field(default=None, description="FK ? routing_rules.id
(nullable)")
290     status: DeliveryStatus = Field(default=DeliveryStatus.PENDING)
291     attempt_count: int = Field(default=0, description="Incremented on each retry")
292     last_error: str | None = None
293     created_at: datetime = Field(default_factory=_utcnow)
294     delivered_at: datetime | None = None
295     next_retry_at: datetime | None = None
296
297
298     class FilterConfig(BaseModel):
299         """Per-user filter configuration.
300
301         v1 rules: duration bounds, minimum height (resolution), and max file size.
302         Stored in the DB as a JSON blob (``filter_config.config_json``) so adding a
303         new field later is backwards-compatible.
304         """
305
306         # Duration bounds in seconds. None means unbounded on that side.
307         min_duration_s: int | None = Field(default=5, ge=0)
308         max_duration_s: int | None = Field(default=7200, ge=0)
309
310         # Resolution: a video passes if its height >= target_height.
311         # None disables the resolution rule.
312         target_height: int | None = Field(default=1080, ge=1)
313
314         # Max file size in mebibytes. None disables the size rule.
315         max_size_mb: int | None = Field(default=2048, ge=1)
316
317         # Content-type exclusions. GIFs and round videos are excluded by default
318         # (their duration/dimensions are usually incidental to real video filters);
319         # silent videos are off by default since some legit clips have no audio.
320         # Existing users' stored configs are unaffected ? the DB JSON blob overrides
321         # these defaults, so this only changes the behaviour for fresh installs.
322         exclude_gif: bool = Field(
323             default=True,
324             description="If True, exclude Telegram GIFs (DocumentAttributeAnimated)",
325         )
326         exclude_round: bool = Field(
327             default=True,
328             description="If True, exclude circular video messages (round_message=True)",
329         )
330         exclude_nosound: bool = Field(
331             default=False,
332             description="If True, exclude videos without an audio track (nosound=True)",
333         )
334
335         # How matched videos are delivered: forward the video, send a link, or both.
336         # Stored in the same JSON blob, so adding this field is backwards-compatible
337         # (existing rows without it default to FORWARD).
338         delivery_mode: DeliveryMode = Field(default=DeliveryMode.FORWARD)
339
340         # Where matched videos / link messages are sent. Accepts:
341         # - "me" (default): Saved Messages ? exact v1.1 behaviour.
342         # - "@username": a public channel the account is a member of.

```

```

343 # - "-100..." or a bare int-as-string: a numeric Telegram chat id
344 # (private channels without a username).
345 # The string is passed verbatim to Telethon's forward_messages / send_message.
346 # See ADR 0005 for rationale and access requirements.
347 delivery_target: str = Field(
348     default="me",
349     min_length=1,
350     description="Telethon entity to deliver to: 'me', '@username', or a chat id",
351 )
352
353 @field_validator("delivery_target", mode="before")
354 @classmethod
355 def _coerce_delivery_target(cls, v: object) -> str:
356     """Accept int chat ids (e.g. -1001234567890) and coerce to str.
357
358     Telethon accepts both str and int as an entity, but storing as str keeps
359     the JSON blob uniform and lets '@username' / 'me' / numeric ids share
360     one field type.
361     """
362     if v is None:
363         return "me"
364     if isinstance(v, int):
365         return str(v)
366     return str(v)
367
368 @model_validator(mode="after")
369 def _check_duration_bounds(self) -> FilterConfig:
370     """Ensure min_duration_s <= max_duration_s when both are set."""
371     if (
372         self.min_duration_s is not None
373         and self.max_duration_s is not None
374         and self.min_duration_s > self.max_duration_s
375     ):
376         raise ValueError(
377             f"min_duration_s ({self.min_duration_s}) must be <= max_duration_s "
378             f"({self.max_duration_s})"
379         )
380     return self
381
382 class VideoRecord(BaseModel):
383     """A seen video message + its metadata. Drives dedup and future /search.
384
385     Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
386     New code should use :class:`ContentItem`; the ``db`` layer maps
387     ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
388     """
389
390     model_config = ConfigDict(frozen=True)
391
392     id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
393     user_id: int
394     channel_id: int = Field(description="Origin Telegram channel id")
395     message_id: int = Field(description="Telegram message id within the channel")
396     document_id: int | None = Field(
397         default=None,
398         description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
399     )
400
401     # Metadata extracted from the Telethon message (may be None if absent).
402     duration_s: int | None = None
403     width: int | None = None
404     height: int | None = None
405     size_bytes: int | None = None

```

```

406         mime_type: str | None = None
407
408         matched: bool = False
409         forwarded: bool = False
410         seen_at: datetime = Field(default_factory=_utcnow)
411
412     @property
413     def dedup_key(self) -> tuple[int, int, int]:
414         """The natural key used for deduplication in the ``videos`` table."""
415         return (self.user_id, self.channel_id, self.message_id)
416
417     @property
418     def content_fingerprint(self) -> tuple[int, int, int, int] | None:
419         """Composite key for Tier-2 content dedup: (size_bytes, duration_s, height,
width).
420
421         Returns None if any field is missing (cannot fingerprint).
422         """
423         if None in (self.size_bytes, self.duration_s, self.height, self.width):
424             return None
425         return (self.size_bytes, self.duration_s, self.height, self.width) # type:
ignore[return-value]
426
427
428     # Default filter config shared by the bootstrap onboarding step.
429     DEFAULT_FILTER_CONFIG = FilterConfig()
430

```

5. assistant (2026-07-06T07:52:32.824Z)

Now let me read the record_delivery and related repo functions in db.py.

6. user (2026-07-06T07:52:34.651Z)

```

1275:async def upsert_destination(conn: aiomysqlite.Connection, dest: Destination) -> Destination:
1307:async def get_destination(
1320:async def upsert_delivery_attempt(
1358:async def record_delivery(
1399:async def get_delivery_attempt(
1613:async def mark_forwarded(conn: aiomysqlite.Connection, video_id: int) -> None:

```

7. user (2026-07-06T07:52:37.437Z)

```

1275     async def upsert_destination(conn: aiomysqlite.Connection, dest: Destination) ->
Destination:
1276         """Insert or update a destination row by (account, provider, target)."""
1277         await conn.execute(
1278             """INSERT INTO destinations
1279                 (account_id, provider, target, kind, title, is_output, created_at)
1280                 VALUES (?, ?, ?, ?, ?, ?, ?)
1281                 ON CONFLICT(account_id, provider, target) DO UPDATE SET
1282                     kind=excluded.kind,
1283                     title=excluded.title,
1284                     is_output=excluded.is_output""",
1285             (
1286                 dest.account_id,
1287                 dest.provider,
1288                 dest.target,
1289                 dest.kind,
1290                 dest.title,
1291                 int(dest.is_output),
1292                 dest.created_at.isoformat(),
1293             ),
1294         )

```

```

1295     await conn.commit()
1296     conn.row_factory = aiosqlite.Row
1297     async with conn.execute(
1298         """SELECT * FROM destinations
1299             WHERE account_id = ? AND provider = ? AND target = ?""",
1300         (dest.account_id, dest.provider, dest.target),
1301     ) as cur:
1302         row = await cur.fetchone()
1303     assert row is not None
1304     return _row_to_destination(row)
1305
1306
1307     async def get_destination(
1308         conn: aiosqlite.Connection, account_id: int, provider: str, target: str
1309     ) -> Destination | None:
1310         """Resolve a destination by its (account, provider, target) natural key."""
1311         conn.row_factory = aiosqlite.Row
1312         async with conn.execute(
1313             "SELECT * FROM destinations WHERE account_id = ? AND provider = ? AND target = ?",
1314             (account_id, provider, target),
1315         ) as cur:
1316             row = await cur.fetchone()
1317             return _row_to_destination(row) if row else None
1318
1319
1320     async def upsert_delivery_attempt(
1321         conn: aiosqlite.Connection, attempt: DeliveryAttempt
1322     ) -> DeliveryAttempt:
1323         """Insert or update a delivery_attempt; idempotent on (content_item,
1324         destination)."""
1325         await conn.execute(
1326             """INSERT INTO delivery_attempts
1327                 (content_item_id, destination_id, route_id, status, attempt_count,
1328                 last_error, created_at, delivered_at, next_retry_at)
1329                 VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
1330                 ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
1331                     status=excluded.status,
1332                     attempt_count=excluded.attempt_count,
1333                     last_error=excluded.last_error,
1334                     delivered_at=excluded.delivered_at,
1335                     next_retry_at=excluded.next_retry_at""",
1336             (
1337                 attempt.content_item_id,
1338                 attempt.destination_id,
1339                 attempt.route_id,
1340                 attempt.status.value,
1341                 attempt.attempt_count,
1342                 attempt.last_error,
1343                 attempt.created_at.isoformat(),
1344                 attempt.delivered_at.isoformat() if attempt.delivered_at else None,
1345                 attempt.next_retry_at.isoformat() if attempt.next_retry_at else None,
1346             ),
1347         )
1348         await conn.commit()
1349         conn.row_factory = aiosqlite.Row
1350         async with conn.execute(
1351             "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id = ?",
1352             (attempt.content_item_id, attempt.destination_id),
1353         ) as cur:
1354             row = await cur.fetchone()
1355             assert row is not None
1356             return _row_to_delivery_attempt(row)

```

```

1357
1358 async def record_delivery(
1359     conn: aiosqlite.Connection,
1360     *,
1361     content_item_id: int,
1362     destination_id: int,
1363     route_id: int | None = None,
1364     success: bool,
1365     error: str | None = None,
1366 ) -> None:
1367     """Record a delivery outcome for one (content_item, destination) pair.
1368
1369     This is the post-ADR-0011 replacement for ``mark_forwarded``. On success,
1370     sets status=delivered; on failure, increments attempt_count and records the
1371     error. Idempotent: repeated calls update the same row.
1372     """
1373     status = DeliveryStatus.DELIVERED if success else DeliveryStatus.FAILED
1374     await conn.execute(
1375         """INSERT INTO delivery_attempts
1376             (content_item_id, destination_id, route_id, status, attempt_count,
1377              last_error, created_at, delivered_at)
1378             VALUES (?, ?, ?, ?, 1, ?, strftime('%Y-%m-%dT%H:%M:%SZ', 'now'),
1379                     CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now') END)
1380             ON CONFLICT(content_item_id, destination_id) DO UPDATE SET
1381                 status=excluded.status,
1382                 attempt_count=delivery_attempts.attempt_count + 1,
1383                 last_error=excluded.last_error,
1384                 delivered_at=CASE WHEN ? THEN strftime('%Y-%m-%dT%H:%M:%SZ', 'now')
1385                             ELSE delivery_attempts.delivered_at END""",
1386         (
1387             content_item_id,
1388             destination_id,
1389             route_id,
1390             status.value,
1391             error,
1392             success,
1393             success,
1394         ),
1395     )
1396     await conn.commit()
1397
1398
1399 async def get_delivery_attempt(
1400     conn: aiosqlite.Connection, content_item_id: int, destination_id: int
1401 ) -> DeliveryAttempt | None:
1402     """Read a delivery attempt row by its natural key (idempotency boundary)."""
1403     conn.row_factory = aiosqlite.Row
1404     async with conn.execute(
1405         "SELECT * FROM delivery_attempts WHERE content_item_id = ? AND destination_id = ?"
1406     ) as cur:
1407         (content_item_id, destination_id),
1408     ) as cur:
1409         row = await cur.fetchone()
1410         return _row_to_delivery_attempt(row) if row else None
1411
1412 # --- legacy Channel shim (backwards-compatible view over sources) -----
1413
1414
1415 async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1416     """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1417
1418     ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1419     reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1420     New code should call :func:`upsert_source` directly.

```

```

1421     """
1422     src = await upsert_source(
1423         conn,
1424         Source(
1425             account_id=channel.user_id,
1426             provider="telegram",
1427             provider_source_id=str(channel.channel_id),
1428             title=channel.title,
1429             kind="channel",
1430             watched=True, # back-compat: pre-ADR-0011 channels were all watched
1431             seen_first_at=channel.seen_first_at,
1432         ),
1433     )
1434     return _source_to_channel(src)

```

8. assistant (2026-07-06T07:52:39.941Z)

Now I have the full picture. Let me verify the BOTH-mode shared-row scenario with a repro. Let me check the schema for delivery_attempts first.

9. user (2026-07-06T07:52:45.103Z)

```

5:for the provider-neutral ``sources`` / ``content_items`` / ``policies`` /
6:``destinations`` / ``routing_rules`` / ``delivery_attempts`` schema (ADR 0011).
38:CREATE TABLE IF NOT EXISTS accounts (
46:CREATE TABLE IF NOT EXISTS telegram_sessions (
55:CREATE TABLE IF NOT EXISTS channels (
64:CREATE TABLE IF NOT EXISTS videos (
81:CREATE TABLE IF NOT EXISTS filter_config (
87:-- ``content_items``, and ``policies`` are populated by migration 003 (and by
88:-- the shims below); ``destinations`` / ``routing_rules`` / ``delivery_attempts``
92:CREATE TABLE IF NOT EXISTS sources (
105:CREATE TABLE IF NOT EXISTS content_items (
128:CREATE TABLE IF NOT EXISTS policies (
143:CREATE TABLE IF NOT EXISTS destinations (
155:CREATE TABLE IF NOT EXISTS routing_rules (
167:CREATE TABLE IF NOT EXISTS delivery_attempts (
169:    content_item_id INTEGER NOT NULL REFERENCES content_items(id),
255:# Pure function: computes the ``content_items.dedup_key`` value from metadata
382:    CREATE TABLE accounts (
403:    CREATE TABLE telegram_sessions (
437:    CREATE TABLE channels (
451:    CREATE TABLE videos (
474:    CREATE TABLE filter_config (
508:    Creates ``sources``, ``content_items``, ``policies``, ``destinations``,
509:    ``routing_rules``, ``delivery_attempts`` (all ``IF NOT EXISTS`` ? on fresh
551:    # 2. Backfill content_items from videos in two phases (review findings #2 & #3):
559:    INSERT INTO content_items
585:    UPDATE content_items SET dedup_key = 'tg:doc:' || document_id
589:    SELECT MIN(id) FROM content_items
599:    UPDATE content_items SET dedup_key =
607:    SELECT MIN(id) FROM content_items
640:    # produce a content_items row. A mismatch signals fingerprint collisions
646:    async with conn.execute("SELECT COUNT(*) FROM content_items") as cur:
653:        "migration_003: %d videos rows produced %d content_items rows "
674:    Also backfills ``delivery_attempts`` rows from the legacy ``videos_deprecated``
727:    # 3. Backfill delivery_attempts from previously-forwarded videos.
730:    INSERT OR IGNORE INTO delivery_attempts
736:    JOIN content_items ci
1158:    """INSERT INTO content_items
1196:    """SELECT * FROM content_items
1215:    "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",

```

[Showing results with pagination = limit: 40]

10. user (2026-07-06T07:52:50.114Z)

```
167 CREATE TABLE IF NOT EXISTS delivery_attempts (
168     id INTEGER PRIMARY KEY AUTOINCREMENT,
169     content_item_id INTEGER NOT NULL REFERENCES content_items(id),
170     destination_id INTEGER NOT NULL REFERENCES destinations(id),
171     route_id INTEGER REFERENCES routing_rules(id),
172     status TEXT NOT NULL DEFAULT 'pending',
173     attempt_count INTEGER NOT NULL DEFAULT 0,
174     last_error TEXT,
175     created_at TEXT NOT NULL,
176     delivered_at TEXT,
177     next_retry_at TEXT,
178     UNIQUE (content_item_id, destination_id)
179 );
180 ""
181
182 # Note: the ``idx_videos_user_document`` partial unique index is created by
183 # migration 001 (below), not in ``_SCHEMA``. It references ``document_id``
184 # which is present on fresh tables but only added to legacy v1 DBs by the
185 # migration; putting it in ``_SCHEMA`` would break ``executescript`` on legacy
186 # DBs that don't yet have the column.
```

11. assistant (2026-07-06T07:52:51.980Z)

Now let me write the repro to confirm the BOTH-mode masking scenario.

12. assistant (2026-07-06T07:53:00.198Z)

```
import asyncio
import aiosqlite
from tg_video_filter import db
from tg_video_filter.models import Destination

async def main():
    conn = await aiosqlite.connect(":memory:")
    await db._apply_schema(conn)
    # seed account, source, content_item, destination
    await conn.execute("INSERT INTO accounts (id, telegram_id, provider, created_at) VALUES (1, 'telegram', '2020-01-01T00:00:00Z')")
    await conn.execute("INSERT INTO sources (id, account_id, provider, provider_source_id, kind, watched, seen_first_at) VALUES (1,1,'telegram','100','channel',1,'2020-01-01T00:00:00Z')")
    await conn.execute("INSERT INTO content_items (id, account_id, source_id, provider, provider_message_id, content_type, dedup_key, seen_at) VALUES (1,1,1,'telegram','42','video','tg:x','2020-01-01T00:00:00Z')")
    await conn.execute("INSERT INTO destinations (id, account_id, provider, target, kind, created_at) VALUES (5,1,'telegram','me','saved','2020-01-01T00:00:00Z')")
    await conn.commit()

    # BOTH mode: forward succeeds, then link fails - same destination_id=5
    await db.record_delivery(conn, content_item_id=1, destination_id=5, success=True)
    print("after forward success:", (await db.get_delivery_attempt(conn, 1, 5)))
    await db.record_delivery(conn, content_item_id=1, destination_id=5, success=False, error="link failed")
    att = await db.get_delivery_attempt(conn, 1, 5)
    print("after link failure:", att)
    print("STATUS:", att.status, "attempt_count:", att.attempt_count, "delivered_at:", att.delivered_at)

asyncio.run(main())
```

13. user (2026-07-06T07:53:00.199Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

14. user (2026-07-06T07:53:10.794Z)

```
after forward SUCCESS: status=delivered count=1 delivered_at=2026-07-06 07:53:10+00:00
after link FAILURE: status=failed count=2 delivered_at=2026-07-06 07:53:10+00:00
Exception in thread Thread-1 (_connection_worker_thread):
Traceback (most recent call last):
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 66, in
_connection_worker_thread
    future.get_loop().call_soon_threadsafe(set_result, future, result)
    ~~~~~^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
878, in call_soon_threadsafe
    self._check_closed()
    ~~~~~^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
556, in _check_closed
    raise RuntimeError('Event loop is closed')
RuntimeError: Event loop is closed
```

During handling of the above exception, another exception occurred:

```
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 1044, in
_bootstrap_inner
    self.run()
    ~~~~~^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 995, in
run
    self._target(*self._args, **self._kwargs)
    ~~~~~^
  File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 75, in
_connection_worker_thread
    future.get_loop().call_soon_threadsafe(set_exception, future, e)
    ~~~~~^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
878, in call_soon_threadsafe
    self._check_closed()
    ~~~~~^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
556, in _check_closed
    raise RuntimeError('Event loop is closed')
RuntimeError: Event loop is closed
```

15. assistant (2026-07-06T07:53:17.650Z)

Confirmed (the trailing exception is just the connection not being closed in my script ? irrelevant to the finding). The repro proves the masking:

- After forward SUCCESS: status=delivered, count=1, delivered_at set
- After link FAILURE: status=**failed**, count=**2**, delivered_at **still set**

So in BOTH mode, a successful forward followed by a failed link leaves a single row saying `status=failed` with a stale `delivered\_at`, and `attempt\_count=2` for one logical delivery. A retry/idempotency consumer reading `status=failed` would re-deliver the already-forwarded video (or misreport it).

Now let me verify the increment/delivered_at semantics in a pure-retry scenario (fail then retry-succeed) to confirm that path is correct, and check the reverse (link succeeds after forward succeeds ? the intended happy path).

16. user (2026-07-06T07:53:30.142Z)

```
fail then retry-success          -> status=delivered count=2 delivered_at=True
last_error=None
BOTH: forward-ok then link-ok    -> status=delivered count=2 delivered_at=True
```

```

last_error=None
BOTH: forward-ok then link-fail      -> status=failed count=2 delivered_at=True
last_error=link
BOTH: forward-fail then link-ok      -> status=delivered count=2 delivered_at=True
last_error=None

```

17. assistant (2026-07-06T07:53:36.518Z)

All scenarios confirmed. Analysis of each:

1. ****fail then retry-success**** ? status=delivered, count=2, delivered_at set ? correct.
2. ****BOTH forward-ok then link-ok**** ? status=delivered, count=2 ? status correct, but count=2 for one logical delivery (over-counts). Minor.
3. ****BOTH forward-ok then link-fail**** ? status=****failed****, count=2, delivered_at ****still True****, last_error=link ? the masking bug. Row claims failed but forward actually succeeded; delivered_at is stale/inconsistent with status.
4. ****BOTH forward-fail then link-ok**** ? status=delivered, count=2 ? the link succeeded so status=delivered is arguably OK, but the forward actually FAILED and that failure is now completely erased (no record the video-forward failed).

Now let me confirm the failure-path coverage claim (lines 145,152,181,224 uncovered) and check test_delivery.py to see what's actually tested for destination_id.

18. user (2026-07-06T07:53:37.506Z)

```

200:         DeliveryMode.BOTH,
251:async def test_deliver_with_destination_id_records_delivery_attempt(
254:     """When destination_id is provided, record_delivery writes a DeliveryAttempt (Track
C)."""
270:         destination_id=dest.id, # type: ignore[arg-type]
272:         # With destination_id, a DeliveryAttempt is written.
283:         """When BOTH mode's link send fails, a warning is logged."""
292:         DeliveryMode.BOTH,
363:         """BOTH mode routes both the forward and the link to the target."""
371:         DeliveryMode.BOTH,

```

19. user (2026-07-06T07:53:42.260Z)

```

245         assert row is not None
246         # forwarded is always False post-ADR-0011 (the column is dropped; delivery
247         # state moves to Track C). The match is still recorded via `matched`.
248         assert row.forwarded is False
249
250
251     async def test_deliver_with_destination_id_records_delivery_attempt(
252         db_with_video: tuple[aiosqlite.Connection, int],
253     ) -> None:
254         """When destination_id is provided, record_delivery writes a DeliveryAttempt (Track
C)."""
255         c, vid = db_with_video
256         # Create a destination row for the test
257         dest = await db.upsert_destination(
258             c, Destination(account_id=USER_ID, provider="telegram", target="me")
259         )
260         assert dest.id is not None
261         client = FakeClient()
262         await deliver(
263             client,
264             _SENTINEL_MSG,
265             c,
266             vid,
267             DeliveryMode.LINK,
268             link_text="test",
269             target="me",

```

```

270         destination_id=dest.id, # type: ignore[arg-type]
271     )
272     # With destination_id, a DeliveryAttempt is written.
273     attempt = await db.get_delivery_attempt(c, vid, dest.id)
274     assert attempt is not None
275     assert attempt.status.value == "delivered"
276     assert attempt.attempt_count == 1
277
278
279     async def test_both_mode_logs_warning_on_link_failure(
280         db_with_video: tuple[aiosqlite.Connection, int],
281         caplog: pytest.LogCaptureFixture,
282     ) -> None:
283         """When BOTH mode's link send fails, a warning is logged."""
284         c, vid = db_with_video
285         # First call (forward) succeeds; second (send_message) raises.
286         client = FakeClient(side_effects=[None, RuntimeError("link fail")])
287         await deliver(
288             client,
289             _SENTINEL_MSG,
290             c,
291             vid,
292             DeliveryMode.BOTH,
293             link_text="test", # type: ignore[arg-type]
294         )
295         assert "link_delivery_failed_in_both_mode" in caplog.text
296
297
298     async def test_link_mode_fallback_to_forward_is_logged(
299         db_with_video: tuple[aiosqlite.Connection, int],
300         caplog: pytest.LogCaptureFixture,
301     ) -> None:
302         """LINK mode with no link_text falls back to FORWARD and logs a warning."""
303         c, vid = db_with_video
304         client = FakeClient()

```

20. assistant (2026-07-06T07:53:48.490Z)

Confirmed: `test\_both\_mode\_logs\_warning\_on\_link\_failure` (line 279) exercises BOTH with forward-ok/link-fail but does **not** pass `destination\_id`, so the failure-recording branches (delivery.py:152, 181) and the masking never get asserted. The only destination_id test (line 251) is LINK-mode success only. So the failure path and the BOTH+destination_id masking are entirely untested.

I have everything I need. Let me finalize findings.

The core finding is the BOTH-mode shared-row masking (confirmed via repro). Related sub-issues: attempt_count over-counts in BOTH mode (2 for one logical delivery even on full success), and forward-failure being erased by a later link-success. These are all latent until Track D wires destination_id into the live path. The failure path is also untested.

21. user (2026-07-06T07:54:20.913Z)

Structured output provided successfully